



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



**FACULTAD
DE INGENIERÍA**

Universidad Nacional de Cuyo

Facultad de Ingeniería

Robótica I

Proyecto Final:

ANÁLISIS CINEMÁTICO DEL ROBOT UR3e DE 6 GRADOS DE LIBERTAD

Integrantes:

Mercado Lombard, Tomás - Leg. 12640

Sotar Maidana, David Maximiliano - Leg. 12231

Vitelli, Gabriel Nicolas - Leg. 13001

2023

Índice

1. Resumen.....	3
2. Introducción.....	3
3. Presentación formal/técnica del robot y la tarea.....	4
3.1. Tarea a realizar.....	4
3.2. Características del robot.....	5
3.3. Actuadores.....	5
3.4. Sensores.....	6
3.4.1. Consideraciones de Seguridad:.....	7
3.5. Espacio de trabajo.....	7
4. Modelado.....	9
4.1. Convención Denavit y Hartenberg.....	9
4.2. Offset y Límites Articulares.....	11
4.3. Definición de la matriz de “base” y “tool”:.....	11
5. Cinemática del robot.....	12
5.1. Cinemática Directa.....	12
5.2. Cinemática Inversa.....	13
6. Relación de Velocidades y puntos singulares.....	14
6.1. Jacobiano y su determinante.....	14
6.2. Puntos singulares.....	14
7. Planificación y Generación de Trayectorias.....	16
7.1. Planificación.....	16
7.2. Generación de Trayectorias.....	18
7.3. Caso de aplicación.....	19
7.4. Entrenamiento del robot.....	19
8. Conclusión y proyecciones a futuro.....	20
8.1. Perspectivas de futuro.....	20
8.2. Conclusiones.....	20
9. Referencias.....	21
Anexos.....	22
1. Función de Cinemática Inversa UR3e.....	22
2. Funciones de generación de trayectoria.....	25

1. Resumen

El proyecto final en Robótica 1 se enfocó en la exploración de la cinemática del robot UR3e de Universal Robot, con el objetivo de asimilar los conceptos esenciales de la materia y aplicarlos en proyectos futuros. Este exhaustivo proceso abordó aspectos claves como la cinemática directa, la cinemática inversa, la velocidad y la relación entre el espacio cartesiano y el espacio articular mediante el uso del Jacobiano. Además, analizamos puntos singulares y planificación de trayectorias. Documentamos detalladamente el proceso, incorporando un análisis de los sensores y actuadores necesarios para el funcionamiento del UR3e.

Palabras clave: cinemática directa, cinemática inversa, análisis de singularidades, planificación de trayectorias, análisis cinemático del robot UR3e.

2. Introducción

En el ámbito de la robótica, el robot UR3e de Universal Robots se erige como el protagonista de nuestro estudio. Diseñado con seis grados de libertad, este manipulador robótico se posiciona como una herramienta versátil y eficiente para diversas aplicaciones.

La elección del UR3e se fundamenta en su capacidad para adaptarse a tareas específicas, brindando soluciones precisas y flexibles. En nuestro caso de aplicación, la incorporación de este robot en procesos de la elaboración de pizzas y en entornos de producción con flujos de trabajo intensos, promete una gran ventaja competitiva ya que además de automatizar el proceso en sí, permite garantizar la eficiencia y repetibilidad, contribuyendo a la mejora del rendimiento y la calidad del producto final, siendo esta de una calidad similar a la del mejor maestro pizzero.

En este trabajo, nos sumergimos en un periodo de tres meses de estudio detallado, documentando el proceso de análisis de la cinemática del UR3e. Además, exploramos la relación entre el UR3e y su entorno, considerando la importancia de los sensores y actuadores para su funcionamiento integral.

Destacamos el hecho de que este enfoque integral nos permitió proporcionar una visión completa del potencial innovador del UR3e en la aplicación elegida.

3. Presentación formal/técnica del robot y la tarea

3.1. Tarea a realizar

Como se mencionó anteriormente, en nuestro proyecto, nos enfocaremos en el estudio detallado del brazo robótico UR3e de Universal Robots, con el objetivo de automatizar procesos específicos en una cocina industrial dedicada a la elaboración de pizzas gourmet. Inspirados por el caso real presentado en el siguiente enlace:

<https://www.dw.com/es/pazzi-el-robot-pizzero/video-62078297>

Seleccionamos el UR3e debido a sus capacidades de carga y dimensiones, considerándolo idóneo para las tareas de manipulación y desplazamiento en la cocina.

La cocina industrial presentada en el video motivación, muestra tres robots divididos en tres grupos de tareas distintas. El brazo que analizaremos se encarga del desplazamiento de la masa de pizza hacia el horno, siempre manteniendo una posición horizontal, y posteriormente retirándola del horno para disponerla sobre la caja en la mesa de cortado. Este enfoque nos permite cuidar la orientación de las herramientas durante los traslados, evitando problemas como la caída de las pizzas y asegurando un producto final impecable.

La herramienta de manipulación seleccionada para la tarea es una pinza adaptable, la cual utilizará herramientas específicas según la tarea a realizar. Adjuntamos una imagen de la herramienta para su referencia.



Figura 1: Pinza adaptable del Robot a analizar

Además, proporcionamos una representación visual de la disposición de la cocina industrial con indicadores en los puntos de interés que nuestro robot deberá alcanzar.

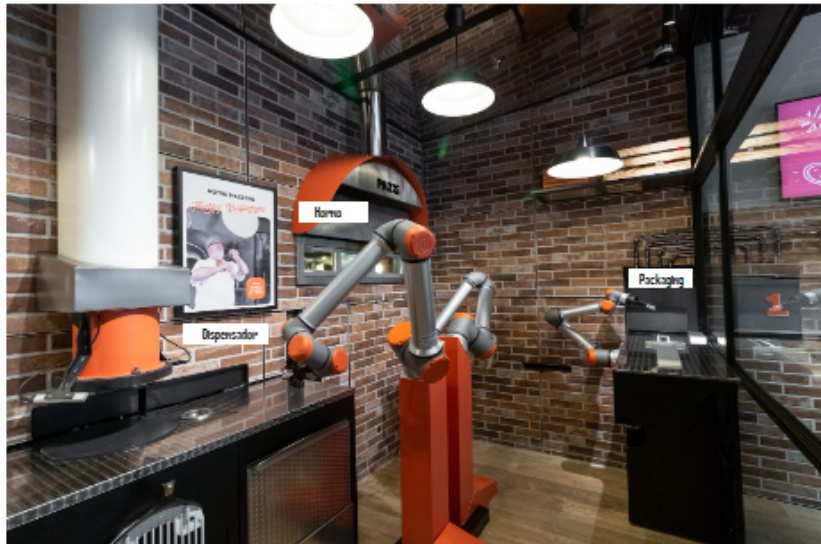


Figura 2: Representación de la disposición real de la aplicación elegida

3.2. Características del robot

El UR3e es un brazo robótico versátil diseñado para diversas aplicaciones industriales, incluida la elaboración de pizzas. Su capacidad de carga de hasta 3 kg, diseño compacto y ligero, y sensores de seguridad lo convierten en una opción eficaz. El robot cuenta con una interfaz de usuario intuitiva y se integra fácilmente en entornos de trabajo reducidos.

El robot se alimenta mediante una caja de control conectada a la red eléctrica monofásica de 110 o 220V. La caja incluye salidas reguladas a 24V y 12V/2A para la alimentación de sensores. El consumo máximo es de aproximadamente 300W, lo que facilita su integración en instalaciones eléctricas estándar.

Es así que el UR3e es un robot comercial muestra un buen nivel de desarrollo y es bien aceptado en la industria, por lo tanto se busca aprovechar de todos los sensores y recursos que son provistos con el robot.

3.3. Actuadores

A la hora de definir o limitar actuadores se necesita profundizar con respecto a la tecnología utilizada en el robot. En nuestro caso, se sugiere que el UR3e utiliza motores síncronos de imán permanente con reducción planetaria, basándose en la

potencia, carga útil y sonidos del robot en funcionamiento. Los drivers utilizados se determinaron observando un detalle eléctrico de la conexión del robot con su caja de control (figura 3).

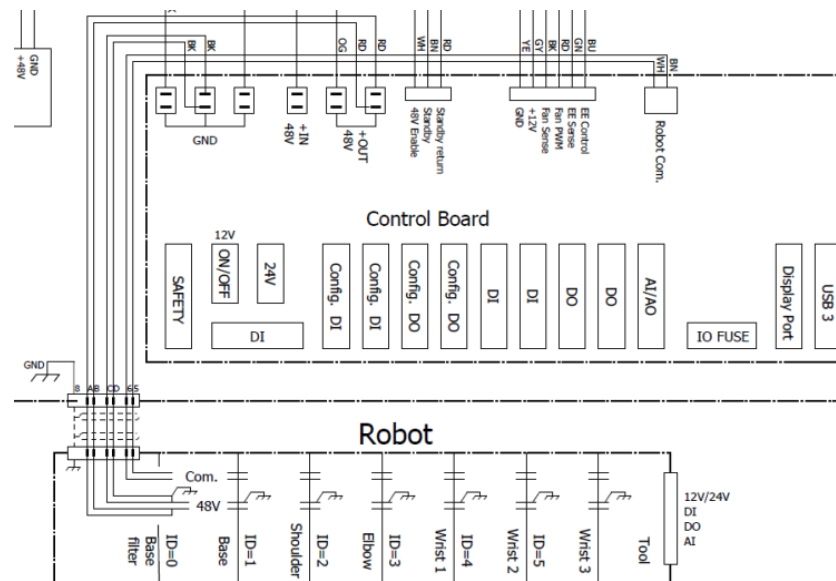


Figura 3: Recorte del diagrama eléctrico UR3e

Es así que evidenciando la poca cantidad de cables que van al robot, se determina que los drivers, presumiblemente PWM para servomotores, se encuentran en el interior del robot y son alimentados con 48V.

3.4. Sensores

Sensores de Fuerza y Torque:

El robot cuenta con sensores de fuerza y torque en cada eje para detectar resistencias a su movimiento. Un sensor de fuerza y torque en la brida final ayuda a identificar el posicionamiento correcto del efector en distintas estaciones. Se sugiere la detección de fuerzas mediante picos de corriente integrados en los drivers de los motores.

Una vez implementados estos sensores, el robot puede verificar la posición adecuada haciendo que el efector colisione contra una superficie plana en ciertas estaciones.

Encoders:

Se asume que el robot utiliza encoders posiblemente ópticos en cada eje. La mención de no mover las articulaciones más de 160° apunta a la presencia de encoders con marcas o 'index'.

Es así que los encoders son cruciales para el posicionamiento preciso del robot. Además se sabe que la información almacenada en una memoria no volátil asegura la retención de la posición tras la calibración inicial.

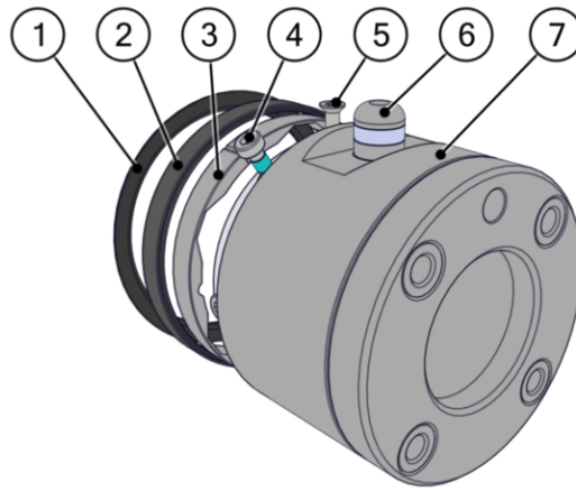


Figura 4: El N°7 de la imagen señala la brida de acople con sensor de torque y fuerza.

3.4.1. Consideraciones de Seguridad:

Sensores de Corriente:

La medición de corriente en los motores puede ayudar a aproximar el torque y detectar posibles fallas o colisiones.

Sensores de Seguridad:

Incorporar sensores que indiquen el estado de la jaula del robot y la presencia de personas en el área de trabajo para garantizar un entorno seguro.

La selección y sugerencia de estos sensores y actuadores se alinean con la aplicación de elaboración de pizzas y la estructura del UR3e, garantizando un rendimiento eficiente y seguro en el proceso.

3.5. Espacio de trabajo

En esta sección, verificamos que el UR3e seleccionado cumple con las dimensiones y el rango de movilidad necesarios para llevar a cabo las tareas descritas. Utilizamos el software Matlab para graficar los espacios de trabajo posibles, considerando diversos parámetros que se detallarán en las próximas secciones de este informe. Es así que logramos determinar el siguiente espacio de trabajo:

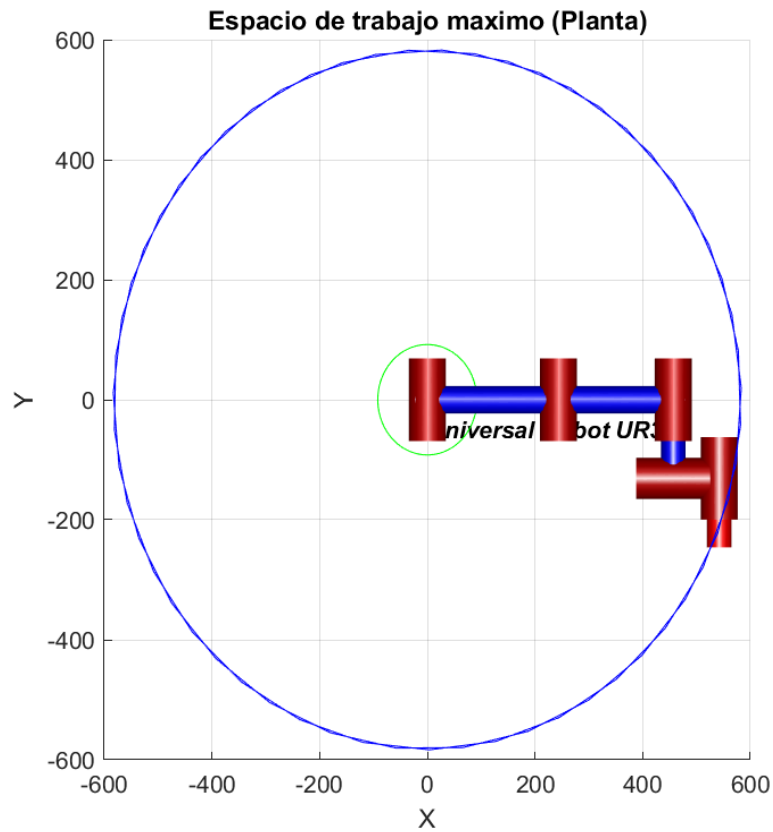


Figura 5: Vista superior del espacio de trabajo de elaboración propia

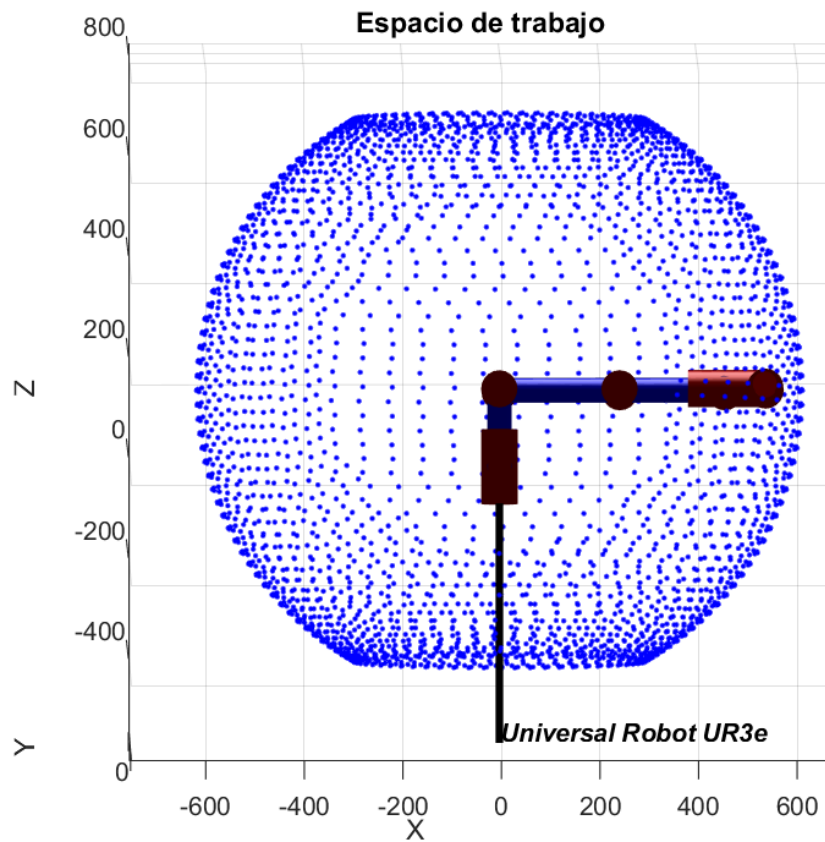


Figura 6: Vista lateral del espacio de trabajo de elaboración propia

Los espacios de trabajo observados coinciden con las especificaciones del fabricante, destacando la limitación diferencial del UR3e en comparación con otros brazos robóticos comerciales, especialmente en el área de trabajo mínimo, representado por una pequeña circunferencia verde en la vista superior. Para rectificar esta coincidencia de espacios de trabajos, se presenta a continuación las especificaciones del fabricante:

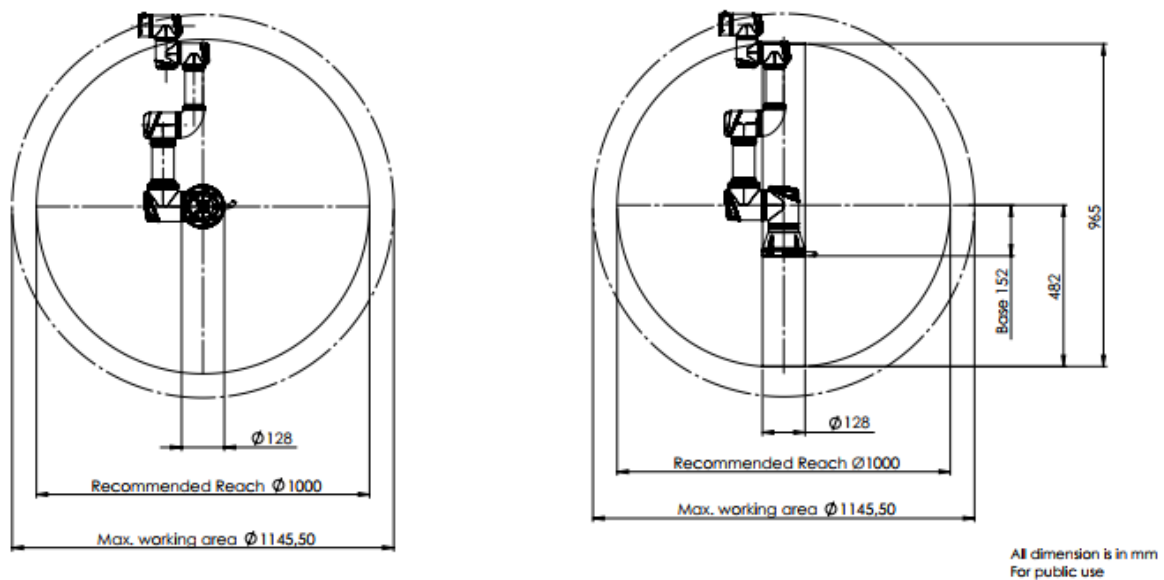


Figura 7: Representación de los espacios de trabajo (Vista superior y Lateral) provista por el fabricante
Tras la observación y análisis de los espacios de trabajo, confirmamos que el UR3e seleccionado se adapta de manera adecuada a las tareas planificadas.

4. Modelado

4.1. Convención Denavit y Hartenberg

Iniciamos nuestro análisis aplicando la convención Denavit-Hartenberg para parametrizar la estructura del UR3e. Los sistemas de articulaciones se definen siguiendo esta convención estándar, como se ilustra en la figura 8.

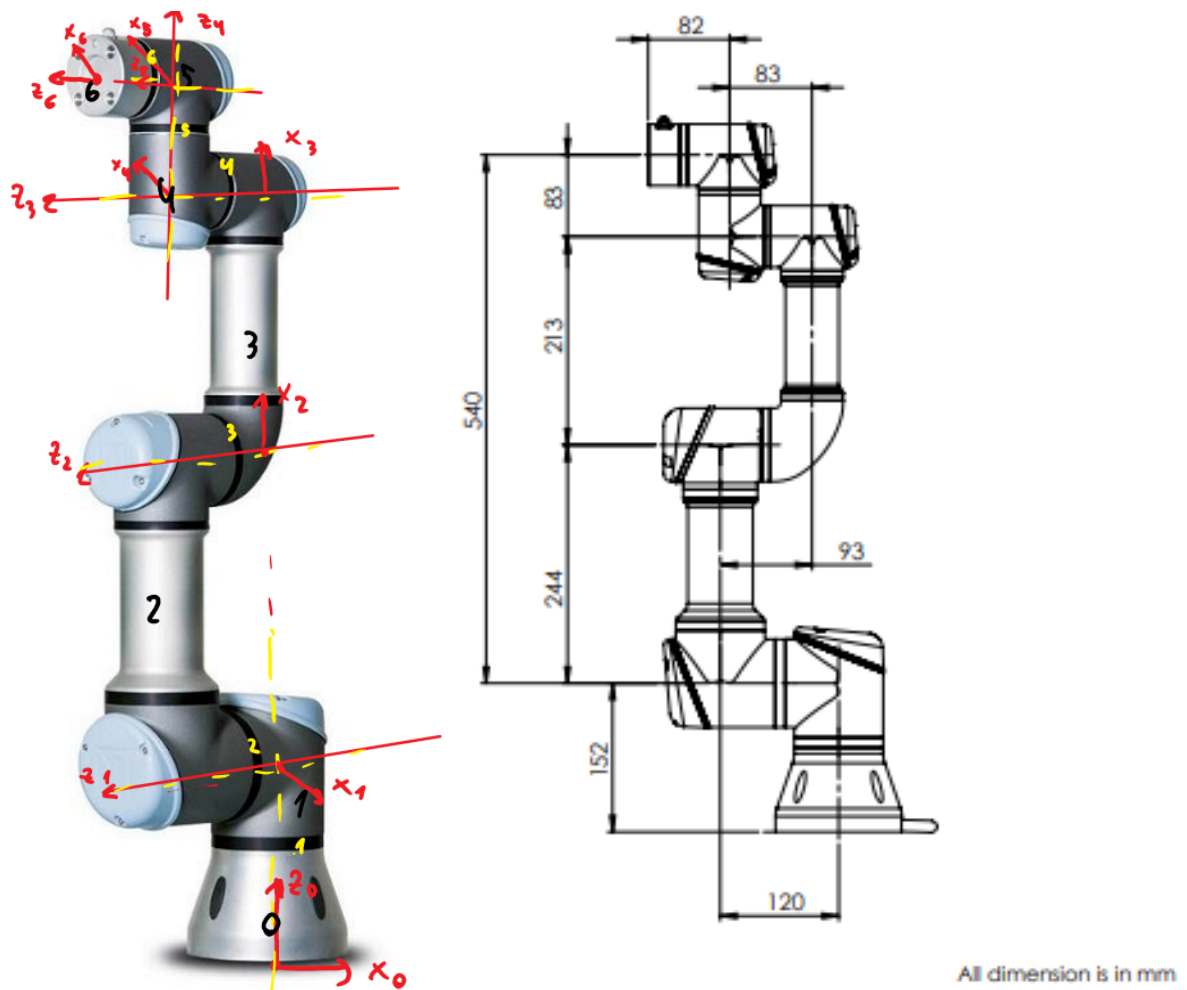


Figura 8: Aplicación de la convención Denavit-Hartenberg en robot UR3e

A partir de estos sistemas, determinamos los parámetros de Denavit-Hartenberg y generamos la correspondiente matriz DH:

Sistema	θ	d	a	α	σ
1	q_1	0	0	$\pi/2$	0
2	q_2	0	243.55	0	0
3	q_3	0	213.2	0	0
4	q_4	131.05	0	$\pi/2$	0
5	q_5	85.35	0	$-\pi/2$	0
6	q_6	92.1	0	0	0

Tabla 1: Matriz Denavit-Hartenberg UR3e

4.2. Offset y Límites Articulares

En nuestro caso de estudio no fue necesario introducir offsets articulares para ajustar el comportamiento del robot.

En relación a los límites articulares, estos se extraen de la ficha técnica y se aplican para garantizar el funcionamiento seguro y eficiente del robot. Los detalles se presentan en la tabla de offsets y límites articulares.

Movimiento de ejes	Rango de trabajo	Velocidad máxima
Base	$\pm 360^\circ$	$\pm 180^\circ /s$
Hombro	$\pm 360^\circ$	$\pm 180^\circ /s$
Codo	$\pm 360^\circ$	$\pm 180^\circ /s$
Muñeca 1	$\pm 360^\circ$	$\pm 360^\circ /s$
Muñeca 2	$\pm 360^\circ$	$\pm 360^\circ /s$
Muñeca 3	Infinito	$\pm 360^\circ /s$

Tabla 2: Límites articulares del UR3e

Estos valores se tuvieron en cuenta para la realización de la función de cinemática inversa y la planificación de trayectorias.

4.3. Definición de la matriz de “base” y “tool”:

En un principio, consideramos el punto de referencia como el origen del robot, estableciendo así la matriz que define la base como la identidad. Pero con un análisis más realista y centrado en la aplicación, encontramos que es más apropiado tomar de referencia plataforma que lo levanta del suelo hasta la altura mínima de una mesada comercial de cocina de 400 mm. Esto además de cumplir con las normas de manejo de alimentos permite que la mayor parte de su área de trabajo se encuentre sobre la mesada.

Teniendo en cuenta que en nuestro caso se seleccionó una mesada de 90 cm de altura. La matriz de base es:

$$R_{\text{base}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{matrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 500 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Para la matriz de tool, se utilizan las medidas del efector final seleccionado. Este es un gripper paralelo modelo “2F-85” de la marca “RobotiQ”.



Figura 9: Imagen representativa del gripper paralelo modelo “2F-85”

Con los planos 2D se puede ubicar el PCL y encontramos que la matriz de tool es:

$$\begin{matrix} R.tool = \\ \begin{matrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & -0.05 \\ 0 & 1 & 0 & 129 \\ 0 & 0 & 0 & 1 \end{matrix} \end{matrix}$$

5. Cinemática del robot

5.1. Cinemática Directa

La cinemática directa consiste en un método utilizado para determinar la posición y orientación del extremo final en el espacio de trabajo a partir de la información de la posición de cada articulación en el espacio de las articulaciones. Este proceso implica aplicar una serie de transformaciones de rotación y traslación a lo largo de todas las articulaciones en la cadena cinemática hasta llegar al extremo final.

Para llevar a cabo la resolución de la cinemática directa, se hace uso del toolbox de robótica. Una vez que se ha definido el objeto "SerialLink" con los parámetros de Denavit-Hartenberg, se utiliza la función "fkine". Esta función toma como entrada un vector de articulación de longitud "n", donde "n" representa el número de articulaciones del robot, y devuelve una matriz de transformación homogénea, que proporciona información acerca de la posición y orientación del extremo final en el espacio cartesiano.

5.2. Cinemática Inversa

Para abordar la cinemática inversa del UR3e, adoptamos un enfoque específico debido a la ausencia de una muñeca esférica. Entre los enfoques o métodos descartados se encontraba el denominado "Método de Pieper" que permite dividir el problema en 2 partes (q_1, q_2, q_3 y q_4, q_5, q_6), porque por la ausencia de dicha muñeca no existe un punto donde se crucen 3 ejes de rotación.

De esta manera el procedimiento para nuestro robot consistió en obtener, en función de la posición y orientación objetivo del efector final, los valores de las coordenadas articulares en el siguiente orden:

1. Se obtuvo el valor de q_1 mediante la posición del efector final y el origen del sistema de coordenadas 'xyz_4', el cual no cambia sin importar el valor de q_6 y q_5 .
2. Se obtuvo el valor de q_5 mediante el valor de q_1 recién obtenido y las componentes del origen del sistema de coordenadas "xyz_5"
3. Se obtuvo el valor de q_6 mediante los valores de q_5 y q_1 , esto se pudo realizar mediante coordenadas esféricas o mediante método matricial. En nuestro caso en particular se optó por desarrollar este último.
4. Una vez obtenidos los valores de q_1 , q_5 y q_6 se puede obtener en orden q_2 , q_3 y q_4 ya que su método de resolución es análogo al caso del robot planar de 3 grados de libertad.

Es relevante destacar que, en la mayoría de los casos, no se obtendrá una única solución, en su lugar serán 8 soluciones distintas. Sin embargo, esto sólo es posible si no tiene en cuenta ni los límites de movimiento de las articulaciones ni los del espacio de trabajo.

Para implementar la cinemática inversa en el robot, se desarrolla una función en MATLAB. Esta función toma como entrada la matriz homogénea que describe la posición y orientación deseada, así como el objeto SerialLink. Con la particularidad de que permite la elección entre todas las soluciones posibles o la más adecuada (“la mejor”) según la posición actual del robot.

Finalmente, la función resultó tener la siguiente estructura:

$$Q = \text{cinv_UR3e}(R, T, q0, \text{mejor})$$

El código de esta se encuentra en el anexo I.

6. Relación de Velocidades y puntos singulares

6.1. Jacobiano y su determinante

En primer lugar, se intentó obtener el jacobiano y su determinante simbólicamente mediante Matlab y Python, pero no fue posible obtener un jacobiano simplificado ni su determinante, ya que este proceso requirió mucho tiempo y excedió la memoria de trabajo de la computadora. Por ello, se optó por realizarlo numéricamente centrándonos en encontrar los puntos singulares en el espacio de trabajo, o en otras palabras, buscar los puntos donde el determinante del jacobiano es igual a cero.

Se desarrolló un algoritmo en Matlab para evaluar puntos aleatorios y detectar singularidades donde el determinante del jacobiano es cero. Siendo posible identificar puntos singulares relacionados con la alineación de ejes, límites del espacio de trabajo y condiciones específicas de los ángulos articulares.

Cabe resaltar que otra opción que se consideró era realizar un barrido ordenado del espacio de trabajo, introduciendo un algoritmo con bucles anidados. Sin embargo, no se optó por esta opción ya que se requeriría un número gigante de evaluaciones, lo cual supera la capacidad de la computadora.

6.2. Puntos singulares

Como se dijo anteriormente, se descubrieron escenarios principales en los que nuestro robot (UR3e) puede tener problemas para moverse a una posición, ya sea porque físicamente no es posible que el robot alcance la posición que requerimos o porque no es posible llegar allí desde la posición actual de las articulaciones del robot. Siento estos los siguientes:

- **Límite de espacio de trabajo exterior:** Es importante recordar que si nos movemos dentro del alcance recomendado, el robot podrá mover la herramienta a cualquier posición con casi cualquier orientación. Pero si necesitamos trabajar en la zona de trabajo máxima, es posible que nuestro robot alcance la mayoría de las posiciones pero con restricciones en la orientación de la herramienta.
- **Límite de espacio de trabajo interior:** En este caso, se vieron que muchas posiciones/orientaciones fueron físicamente incansables cuando se realizaban movimientos del robot en la columna que se encuentra directamente encima y debajo de la base del robot y en puntos cercanos a estos..
- **Singularidades de hombro:** Estas singularidades se dan cuando determinados valores de θ_2 , θ_3 , θ_4 provocan que el efector final no puede ser movido a lo largo del eje z_6 .
- **Singularidades de la muñeca:** Estas singularidades se presentan cuando el ángulo $\theta_5 = 0$ o $\theta_5 = \pm\pi$. Ya que en estos casos, se observa que el z_4 y z_6 se encuentran paralelos.
- **Singularidades del codo:** Estas singularidades se presentan cuando el ángulo $\theta_3 = 0$ o $\theta_3 = \pm\pi$. Esto significa que el brazo está completamente estirado o doblado; sin embargo solo el primer caso es físicamente posible.

Se adjunta las siguientes imágenes representativas de los escenarios identificados:

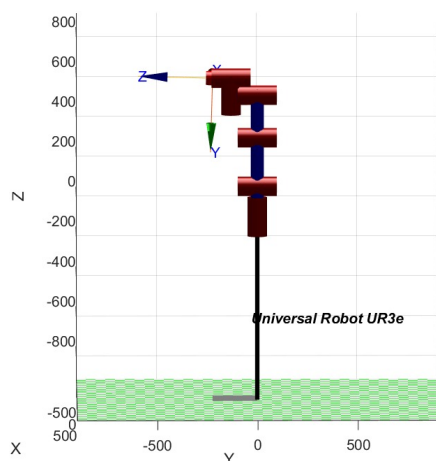


Figura 10: Límite de espacio de trabajo interior

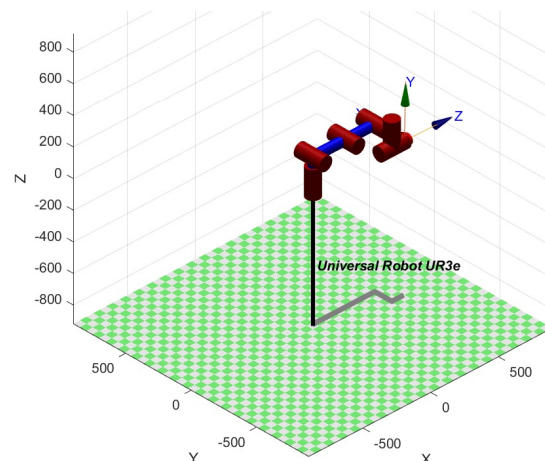


Figura 11: Límite de espacio de trabajo exterior

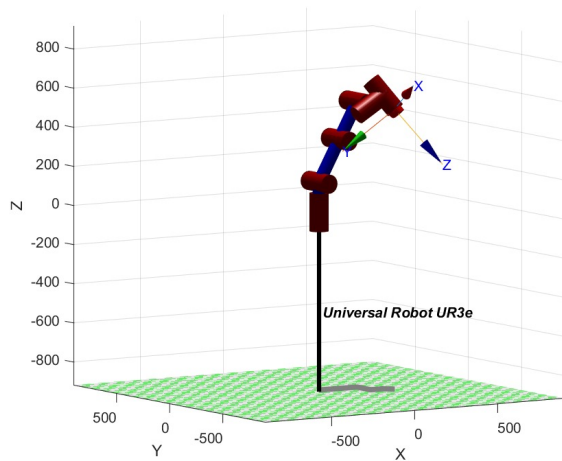


Figura 12: Singularidad del codo ($\theta_3 = 0$)

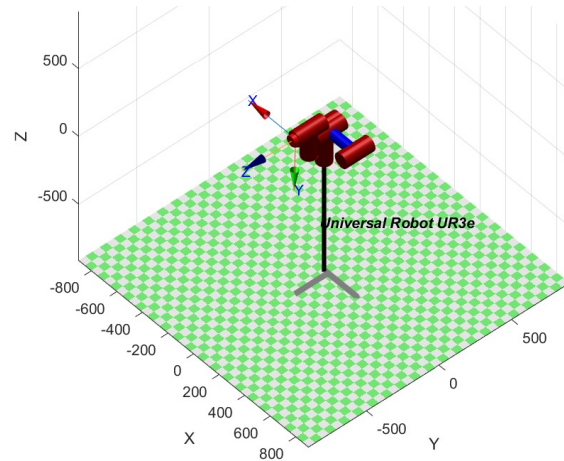


Figura 13: Singularidad del codo ($\theta_3 = \pm\pi$)

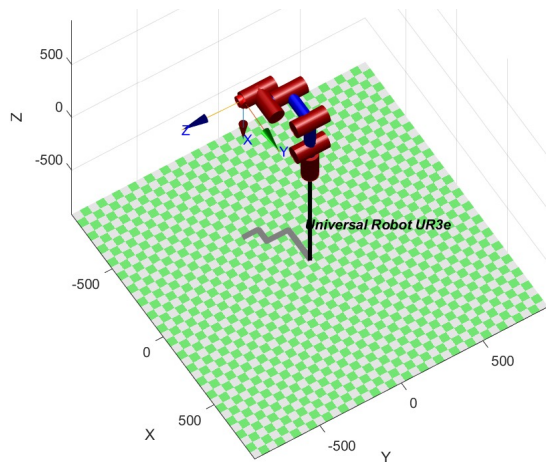


Figura 14: Sing. de la muñeca ($\theta_5 = 0$)

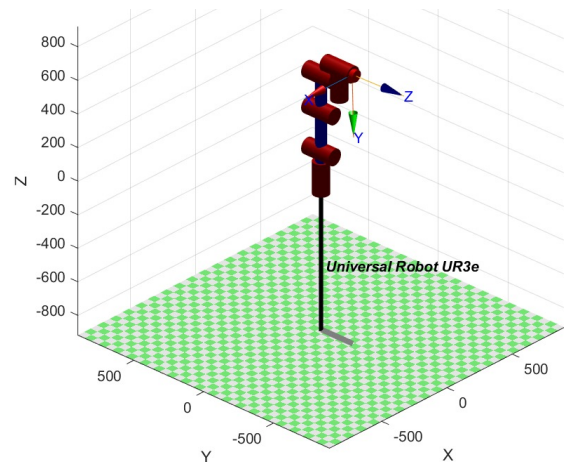


Figura 15: Sing.del hombro ($\theta_2 = \pi/2$, $\theta_3 = 0$, $\theta_4 = \pi/2$)

Finalmente, se destaca que una vez identificadas las singularidades, algo a recordar es que no son los únicos puntos críticos, ya que siempre hay que tener en cuenta los límites articulares.

7. Planificación y Generación de Trayectorias

7.1. Planificación

Para la aplicación de elaboración de pizzas, se diseñó una celda de trabajo considerando las capacidades del robot UR3e. Se ubicaron puntos de entrada y salida estratégicos para el efector, y se generaron trayectorias para movimientos específicos, como tomar la pala, manipular la pizza y colocarla en el horno. Finalmente se destacan los puntos de interés y se presenta una simulación de la trayectoria.

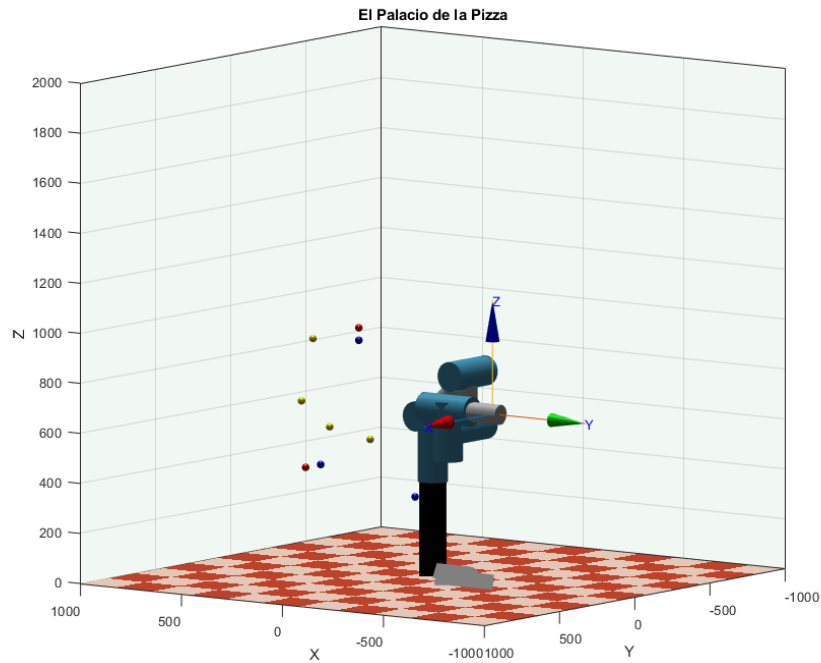


Figura 16: Puntos de interés en la planificación

En la figura 16 se observan los **puntos azules**, los cuales indican el punto de entrada del efector a la estación. Ordenándolos de arriba hacia abajo encontramos:

- Ingreso para tomar la pala
- Ingreso y egreso del horno
- Ingreso y egreso zona de colocación y extracción de la pizza.

Los **puntos amarillos** son puntos intermedios, a los cuales el robot debe acercarse al realizar algunas trayectorias. Y finalmente los **puntos rojos** indican hasta donde debe ingresar el efector del robot en línea recta.

Al estar colocado el robot en una celda cerrada no hay requerimiento de velocidades máximas, pero estas sí se deben tener en cuenta al considerar la dinámica de la pala y la masa de pizza que estará sobre esta. Por otro lado, también es importante comprobar que los movimientos de la pala con la pizza arriba sean de tal forma que esta se mantenga horizontal.

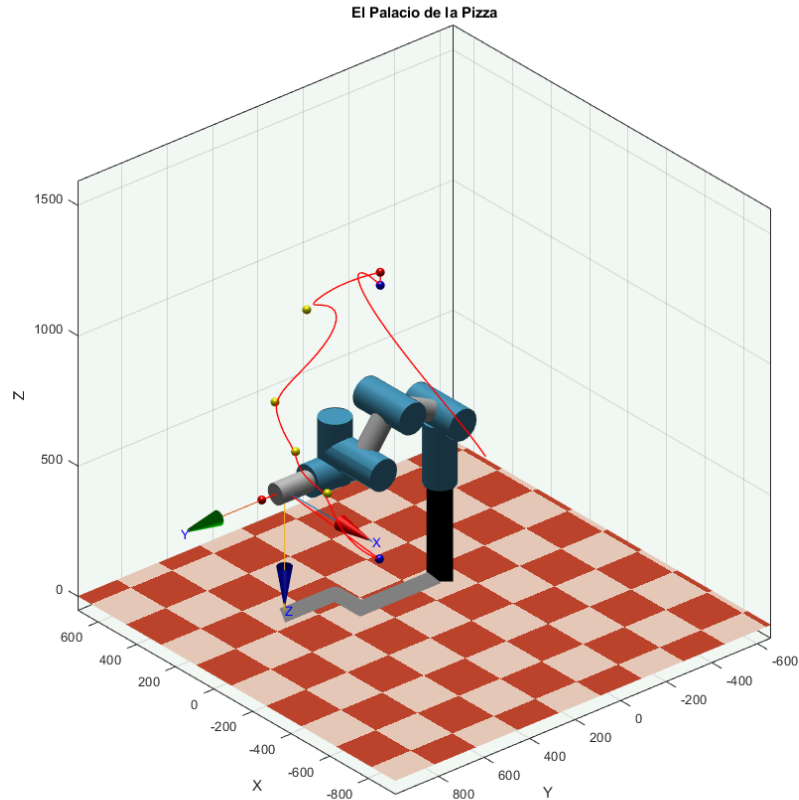


Figura 17: Trayectoria generada para hornear la pizza.

7.2. Generación de Trayectorias

Para la generación de trayectorias de movimiento general del robot y para movimientos que cumplan con los requisitos especificados se implementaron las siguiente 3 funciones:

- **“mover_a”**: Realiza interpolación articular con **‘jtraj’** desde la posición actual hacia la posición indicada.
- **“mover_linea”**: Realiza interpolación en el espacio cartesiano con la función **‘ctrjaj’** y luego utiliza la función anterior para mover el robot entre la pose actual y la pose indicada.
- **“mover_via_a”**: Esta función recibe una lista de poses intermedias, convierte a cada una al espacio articular teniendo en cuenta la solución articular, que produce el menor cambio de los “q” entre pose y pose, provista por la función de cinemática inversa implementada anteriormente. Luego realiza interpolación articular con la función **‘mstraj’** la cual busca acercarse a los puntos intermedios pero sin detenerse en ellos.

7.3. Caso de aplicación

En el caso del movimiento del punto azul “Ingreso y egreso zona de colocación y extracción de la pizza” hacia el punto azul “Ingreso y egreso del horno” se puede utilizar la función “mover_a” o la función “mover_linea”.

En la siguiente figura se observan gráficas con los valores de posición, velocidad y aceleración para ambos métodos de generación de trayectorias:

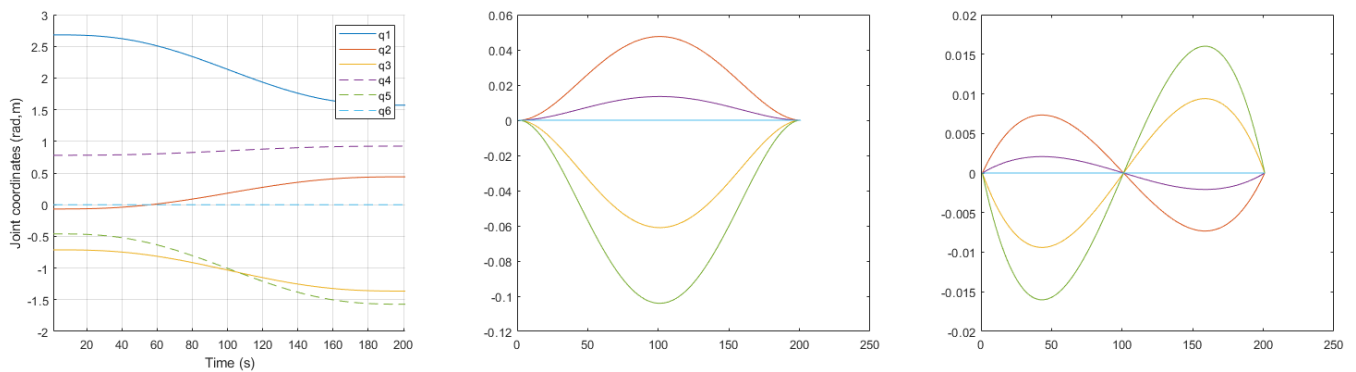


Figura 18: Interpolación Articular

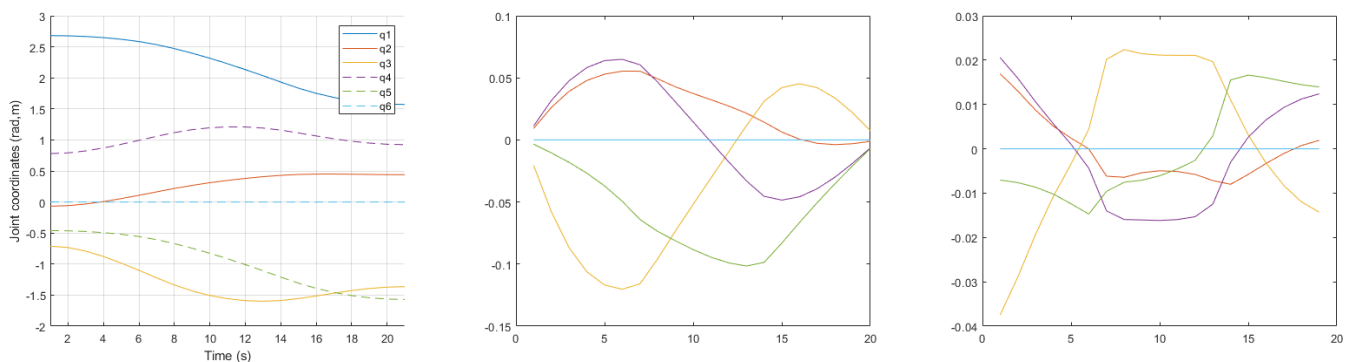


Figura 19: Interpolación Cartesiana

Se compararon entonces los dos métodos de generación de trayectorias: interpolación articular y cartesiana. Observando la variación de los ‘q’ en ambos métodos podemos concluir que la interpolación cartesiana mostró curvas de aceleración complejas y cálculos innecesarios para variaciones mínimas de las coordenadas articulares, siendo contraproducente decidírnos por ese método.

7.4. Entrenamiento del robot

El método desarrollado para el entrenamiento del robot es mediante un sistema de “recetas” las cuales utilizan las funciones implementadas anteriormente para mover

el robot entre los puntos deseados. Es decir, que luego de un análisis de estos métodos expresados en funciones, se optó por implementar una combinación de ellas.

Para definir los puntos se desarrolló un programa llamado “*Entrenador*” el cual permite posicionar el robot en el punto y orientación deseados y luego almacenar ambas en una lista. Este programa es suavemente útil cuando se desea definir puntos intermedios para la realización de trayectorias con la función “**mover_via_a**”.

En la sección de anexos, se encuentran los códigos mencionados en esta sección.

8. Conclusión y proyecciones a futuro

Durante el estudio y análisis cinemático del robot UR3e para la aplicación en la elaboración de pizzas, se han obtenido resultados significativos que destacan la viabilidad y eficiencia de este robot en entornos específicos. A continuación, se presentan las conclusiones más relevante y posibles aplicaciones futuras:

8.1. Perspectivas de futuro

Considerando la naturaleza colaborativa del UR3e, se sugiere explorar aún más la integración del robot en el mismo espacio que los trabajadores humanos y así lograr la sinergia entre las capacidades humanas y las capacidades del robot.

Como trabajo futuro, se podría ampliar la funcionalidad del UR3e para abordar más tareas específicas en la elaboración de pizzas, como la aplicación de ingredientes, el monitoreo de la cocción y la gestión logística en la cocina.

Finalmente, se considera que se podría integrar sistemas de visión y aprendizaje automático podría mejorar la capacidad del robot para reconocer y adaptarse a diferentes situaciones en tiempo real. Esto contribuiría a la versatilidad del UR3e en un entorno dinámico de cocina.

8.2. Conclusiones

En primera instancia, se logró desarrollar un algoritmo eficiente y efectivo para la resolución de la cinemática inversa del UR3e. Se validó que este paso es fundamental para controlar con precisión el movimiento del robot en la elaboración de pizzas.

Posteriormente, la exploración de la relación de velocidades permitió identificar puntos singulares, proporcionando una comprensión más profunda de las limitaciones del robot.

Finalmente, nos encantaría destacar que se estableció una trayectoria suave y eficiente para el robot, demostrando la capacidad de aplicar conceptos teóricos en la práctica. Demostrando que la planificación cuidadosa de trayectorias es sumamente importante para la ejecución exitosa de tareas específicas, como en nuestro caso, la manipulación de ingredientes y utensilios en la elaboración de pizzas.

En conclusión, la combinación de conocimientos teóricos y prácticos nos ha enriquecido la comprensión de la robótica aplicada en contextos específicos, y proporcionado una visión integral de las capacidades del UR3e en la elaboración de pizzas, destacando su potencial y planteando posibles direcciones para investigaciones y desarrollos futuros.

9. Referencias

1. *Universal robots - What is a singularity?* (s. f.).
<https://www.universal-robots.com/articles/ur/application-installation/what-is-a-singularity/>
2. Chen, F., Ju, H., Wang, K., & Cai, N. (2023). An analytical approach based on Dixon resultant for the inverse kinematics of 6R robot manipulators with offset wrists. *Communications in Nonlinear Science and Numerical Simulation*, 127, 107541. <https://doi.org/10.1016/j.cnsns.2023.107541>
3. Arenas-Rosales, F., Martell, F., Sanchez, I. Y., & Paredes-Orta, C. A. (2023). Virtual UR5 Robot for Online Learning of Inverse Kinematics and Independent Joint Control Validated with FSM Position Control. *Robotics*, 12(1), 23. <https://doi.org/10.3390/robotics12010023>
4. Ramírez, D. P., Camacho, J. P. R., & Marín, J. G. G. (2021). *UR3 modelo cinemático inverso*. <https://doi.org/10.15332/dt.inv.2021.02849>
5. *Inverse Kinematic Analysis of Robot Manipulators - CORE*. (n.d.).
<https://core.ac.uk/works/39631552>
6. *MATLAB Documentation - MathWorks América Latina*. (n.d.).
<https://la.mathworks.com/help/matlab/>

Anexos

1. Función de Cinemática Inversa UR3e

```
function Q= cinv_UR3e(R,T,q0,mejor)
%CINV_UR3E Cinematica inversa robot UR3e
% cinv_UR3e(R,T,q0,mejor) Funcion que permite calcular las 8 soluciones
% articulares para que el robot tome una pose especificada.
% si mejor es = 1 devuelve la posicion que implca un menor cambio de los
% q desde q0.

if nargin == 3
    mejor = false;
elseif nargin ~= 4
    error('Argumentos incorrectos')
end
if class(T) ~= "SE3"
    T = SE3(T);
end

T=R.base.inv*T*R.tool.inv;

%q1
q1=calcular_q1(R,T);
qq(1,1:4)=[1 1 1 1]* q1(1);
qq(1,5:8)=[1 1 1 1]* q1(2);

%q5:
q5_1= calcular_q5(R,T,q1(1));
q5_2= calcular_q5(R,T,q1(2));
qq(5,1:2) = [1 1]* q5_1(1);
qq(5,3:4) = [1 1]* q5_1(2);
qq(5,5:6) = [1 1]* q5_2(1);
qq(5,7:8) = [1 1]* q5_2(2);

%q6
for i=1:8
    qq(6,i)=calcular_q6(R,T,qq(1,i),real(qq(5,i)));
end

%q2
for i=1:2:8
    qq(2,[i,i+1])= calcular_q2(R,T,qq(1,i),(qq(5,i)),(qq(6,i)));
end

%q3
for i=1:8
    qq(3,i)=calcular_q3(R,T,qq(1,i),qq(2,i),(qq(5,i)),(qq(6,i)));
end
```

```

%q4
for i=1:8
    qq(4,i)=calcular_q4(R,T,qq(1,i),qq(2,i),qq(3,i),(qq(5,i)),(qq(6,i)));
end

qq=qq - R.offset' * ones(1,8);

if mejor
    Qaux = qq - q0' * ones(1,8);
    normas = zeros(1,8);
    for i=1:8
        normas(i) = norm(Qaux(:,i));
    end
    [~,pos] = min(normas);
    Q = qq(:, pos)';
else
    Q = qq';
end
end

function q=calcular_q1(R,T)
p50=T.t - R.d(6)*T.a;
phi1=atan2(p50(2),p50(1));
phi2=acos(R.d(4)/norm(p50(1:2)));
q(1)=phi1 - phi2 + pi/2;
q(2)=phi1 + phi2 + pi/2;
q=q - (q>pi) *2*pi;
end

function q=calcular_q5(R,T,q1) %funciona = de las 2 formas
%matricial

o1=T.o(1);
o2=T.o(2);
%o3=T.o(3);
n1=T.n(1);
n2=T.n(2);
%n3=T.n(3);
a1=T.a(1);
a2=T.a(2);
%a3=T.a(3);

D= cos(q1)* o2- sin(q1)*o1;
E= sin(q1)* n1- cos(q1)* n2;
q(1)= atan2(sqrt(E^2+D^2),sin(q1)*a1-cos(q1)*a2);
q(2)= -atan2(sqrt(E^2+D^2),sin(q1)*a1-cos(q1)*a2);
q=q - (q>pi)*2*pi;
end

```

```
function q=calcular_q6 (R,T,q1,q5)
```

```
o1=T.o(1);
o2=T.o(2);
%o3=T.o(3);
n1=T.n(1);
n2=T.n(2);
%n3=T.n(3);
```

```
D= cos(q1)* o2- sin(q1)*o1;
E= sin(q1)* n1- cos(q1)* n2;
q=atan2(D/sin(q5),E/sin(q5));
```

```
q=q -(q>pi) * 2 * pi;
end
```

```
function q= calcular_q2(R,T,q1,q5,q6)
```

```
T10=R.links(1).A(q1);
T54=R.links(5).A(q5);
T65=R.links(6).A(q6);
T41=T10.inv*T*T65.inv*T54.inv;
Beta=atan2(T41.t(2),T41.t(1));
L=sqrt(T41.t(1)^2+T41.t(2)^2);
Alfa= acos((R.a(2)^2 + L^2 - R.a(3)^2)/(2*R.a(2)*L));
q(1,1)=Beta+Alfa;
q(1,2)=Beta-Alfa;
q=q - (q>pi)*2*pi;
end
```

```
function q= calcular_q3(R,T,q1,q2,q5,q6)
```

```
T10=R.links(1).A(q1);
T21=R.links(2).A(q2);
T54=R.links(5).A(q5);
T65=R.links(6).A(q6);
T42=T21.inv*T10.inv*T*T65.inv*T54.inv;
a= real(T42.t(2));
b=real(T42.t(1));
```

```
q=atan2(a,b);
q=q - (q>pi)*2*pi;
end
```

```
function q= calcular_q4(R,T,q1,q2,q3,q5,q6)
```

```
T10=R.links(1).A(q1);
T21=R.links(2).A(q2);
T32=R.links(3).A(q3);
T54=R.links(5).A(q5);
T65=R.links(6).A(q6);
```

```

T43=T32.inv*T21.inv*T10.inv*T*T65.inv*T54.inv;
a= real(T43.n(2));
b=real(T43.n(1));
q=atan2(a,b);
q=q - (q>pi)*2*pi;
end

```

2. Funciones de generación de trayectoria

```

function q_final = mover_a(R,q_actual,T1,tiempo)
%MOVER_A Ir interpolando articularmente
% MOVER_A(q_actual,T1,tiempo) Lleva el robot UR de su posición actual al punto
% seleccionado utilizando interpolación articular
if isequal(size(T1),[1,6])
    q_final = T1;
else
    %q_final = R.ikine(T1,'q0',q_actual); % CI por metodos numericos
    q_final = cinv_UR3e(R,SE3(T1),q_actual,1); % Metodo nuestro
end

```

```

tray_Q0_Q1 = jtraj(q_actual,q_final,0:0.1:tiempo);
R.animate(tray_Q0_Q1);
end

```

```

function q_final = mover_linea(R,q_actual,T1,T2,tiempo)
%MOVER_LINEA Mueve el robot en linea recta
% mover_linea(R,q_actual,T1,T2,tiempo) Permite mover el robot en linea
% recta entre las poses indicadas.
T_aux = R.fkine(q_actual).double;
if abs(T_aux-T1) > eps
    % No estoy en el punto inicial
    q_actual = mover_a(R,q_actual,T0,3);
end
dt = tiempo/20;
tray_T1_T2 = ctraj(T1,T2,lspb(0, 1, (0:dt:tiempo)')); %Interpolacion en esp cartesiano

%tray_art = R.ikine(tray_T1_T2,'q0',q_actual); %Paso al espacio articular
tray_art = ones(length(tray_T1_T2),6);

for i=1:length(tray_T1_T2)
    Taux = tray_T1_T2(:,i);
    tray_art(i,:) = cinv_UR3e(R,Taux,q_actual,1); % Metodo nuestro
end

q_final = tray_art(end,:);
try
    R.animate(tray_art);
catch
end

```

end

```
function q_final = mover_via_a(R,qdmax,q_actual,T1,T_vias,tiempo)
%MOVER_VIA_A Movimiento pasando por puntos indicados
% mover_via_a(R,qdmax,q_actual,T1,T_vias,tiempo) Mueve el robot hasta la
% pose final utilizando interpolacion articular y acercandose a las
% posiciones intermedias que se le proveen mediante "T_vias".
if isequal(size(T_vias),[4 4])
    n= 1;
else
    n= length(T_vias);
end
% Se convierte cada punto 'via' a coordenadas q:
vias = ones(n,6);
for i=1:n
    if class(T_vias(i)) == "SE3"
        Taux = T_vias(i).double;
    else
        Taux = T_vias(:,i);
    end
    vias(i,:) = cinv_UR3e(R,Taux,q_actual,1);
end

%q_final = R.ikine(T1,'q0',q_actual); % CI por metodos numericos
q_final = cinv_UR3e(R,SE3(T1),vias(n,:),1); % Metodo nuestro

vias = vertcat(vias,q_final);

tray_Q0_Q1 = mstraj(vias,qdmax,[],q_actual,0.1,1);

R.animate(tray_Q0_Q1);
end
```